ADDENDUM · V1.0 · 2026-11

Addendum au guide

Corrections vérifiées sur la documentation officielle cTrader + cahier des charges de la crate `ctrader_client/`. Document à lire conjointement avec le guide principal (ctrader-rust-arbitrage-guide.pdf, 42 pages).

SECTION 1 · ERRATA

Corrections du guide principal

Trois corrections à appliquer au guide principal *ctrader-rust-arbitrage-guide.pdf*, vérifiées sur la documentation officielle Spotware (`help.ctrader.com/open-api`) et le dépôt `raul-gherman/ctrader-openapi-rust`. Ces corrections sont critiques : la deuxième (unité de volume) provoquerait un envoi d'ordre 100× trop large si codée telle quelle.

1.1 — Endpoints séparés démo / live

Le guide principal (chapitre 2) indique un endpoint générique `ctraderapi.com:5035`. La réalité vérifiée sur la doc officielle : les environnements démo et live utilisent des **hôtes séparés**, jamais mélangés sur la même connexion. Le port 5035 transporte le protobuf binaire ; le port 5036 (non utilisé ici) transporte une variante JSON.

Hôte	Port	Protocole	Usage
demo.ctraderapi.com	5035	Protobuf binaire	Compte démo IC Markets
live.ctraderapi.com	5035	Protobuf binaire	Compte réel IC Markets
ctraderapi.com (générique)	—	—	⚠ NE PAS UTILISER — hôte non routable

Tableau 1 — Endpoints IC Markets corrigés (vérifiés sur doc officielle)

RUST · SRC/CONN.RS — ENDPOINTS

```rust
// src/conn.rs — endpoints corrigés
pub const HOST_DEMO: &str = "demo.ctraderapi.com";
pub const HOST_LIVE: &str = "live.ctraderapi.com";
pub const PORT: u16 = 5035;  // protobuf binaire
// Configuration par environnement
pub enum Env { Demo, Live }
impl Env {
    pub fn host(&self) -> &'static str {
        match self {
            Env::Demo => HOST_DEMO,
            Env::Live => HOST_LIVE,
        }
    }
}
```

CRITIQUE · SÉPARATION DÉMO / LIVE

Ne jamais utiliser `ctraderapi.com` générique. Toujours sélectionner `demo.` ou `live.` explicitement via la configuration. Une erreur d'hôte provoque un échec de connexion silencieux (timeout TLS) ou pire, l'envoi d'ordres réels sur un contexte démo (si l'hôte live répond par défaut).

1.2 — Unité de volume : erreur ×100 dans le guide

Le guide principal (chapitre 9, tableau 5) indique : *« volume en centièmes de lot : 100 000 = 1.0 lot (100k unités) »*. Ceci est **incorrect d'un facteur 100**. La spécification officielle Spotware est sans ambiguïté :

CRITIQUE · VOLUME = CENTIÈMES D'UNITÉ DE BASE
Le champ `volume` de `ProtoOANewOrderReq` est exprimé en **centièmes d'unité de base** (i.e. centièmes de 1 unité de la devise de base). Conversion : `1 000 = 10 unités`, donc `1 lot (100 000 unités) = 10 000 000` dans le champ volume. Coder la table du guide telle quelle causerait l'envoi d'un ordre **100 fois trop grand**.

Volume champ (int64)	Unités de base	Lots FX	Commentaire
1 000	10	0.0001	Micro lot — trop petit pour IC Markets
100 000	1 000	0.01	Volume minimum démo recommandé
1 000 000	10 000	0.1	Volume standard démo
10 000 000	100 000	1.0	1 lot standard — volume live typique
100 000 000	1 000 000	10	Volume max — à éviter sans accord broker

Tableau 2 — Conversion volume (corrigé, vérifié sur doc officielle)

RUST · SRC/ORDERS.RS — VOLUME HELPERS

```rust
// src/orders.rs — helpers volume corrigés
pub const UNIT_BASE: i64 = 1;          // 1 unité de la devise de base
pub const CENTI: i64 = 100;            // 1 centième = 100 unités (volume champ)
pub const LOT: i64 = 10_000_000;       // 1.0 lot = 100k unités × 100 (centièmes)
pub fn lots_to_volume(lots: f64) -> i64 {
    (lots * LOT as f64).round() as i64
}
pub fn volume_to_lots(volume: i64) -> f64 {
    volume as f64 / LOT as f64
}
// Tests
#[test]
fn test_volume_conversion() {
    assert_eq!(lots_to_volume(0.01), 100_000);      // 0.01 lot démo
    assert_eq!(lots_to_volume(1.0), 10_000_000);    // 1.0 lot standard
    assert_eq!(volume_to_lots(10_000_000), 1.0);
    // Vérification : 1 lot = 100 000 unités × 100 centièmes
    assert_eq!(LOT, 100_000 * 100);
}
```

1.3 — LIMIT IOC confirmé (vs MARKET_RANGE)

Le guide principal (chapitre 9 et 11) présuppose l'envoi d'ordres `MARKET` avec exécution parallèle des trois jambes via `tokio::join!`. Pour le moteur `utilustrader.io`, cette sémantique est **remplacée** par des ordres `LIMIT IOC` séquentiels par jambe, conformément au backtest validé. La confirmation vient de l'énumération officielle `ProtoOATimeInForce` :

Constante ProtoOATimeInForce	Valeur	Sémantique
GOOD_TILL_CANCEL	1	Ordre reste actif jusqu'annulation manuelle
IMMEDIATE_OR_CANCEL	2	IOC — remplit ce qui est possible, annule le reste
FILL_OR_KILL	3	FOK — tout ou rien (alternative stricte)
MARKET_ON_OPEN	4	Exécution à l'ouverture du marché suivant
GOOD_TILL_DATE	5	Expire à une date donnée (champ expirationTimestamp)

Tableau 3 — ProtoOATimeInForce (vérifié sur doc officielle)

Le guide mentionnait également `MARKET_RANGE` avec `slippageInPoints` comme variante d'ordre marché à slippage borné. Bien que cette option existe dans l'API, elle est **écartée** du cahier des charges `ctrader_client/` au profit de `LIMIT IOC` strict, pour préserver la parité avec le backtest validé. La table de mapping ci-dessous (section 3) formalise ce choix.

SECTION 2 · CAHIER DES CHARGES

Crate ctrader_client/ — architecture cible

Crate sœur du moteur principal, dédiée à l'encapsulation du client cTrader. Responsabilité unique : gérer la connexion, l'authentification, le flux de ticks et l'envoi d'ordres LIMIT IOC. Aucune logique de stratégie ici — la détection d'opportunité et le calcul des jambes restent dans le module `engine` du moteur principal.

2.1 — Structure des modules

TEXT · ARBORESCENCE CTRADER_CLIENT/

```
ctrader_client/
├── Cargo.toml
├── src/
│   ├── lib.rs            # exports publics : Client, Env, OrderResult
│   ├── proto/
│   │   └── spotware.rs   # fichier du dépôt raul-gherman (tel quel)
│   ├── conn.rs           # TLS rustls + TCP_NODELAY + codec 4B-BE
│   ├── auth.rs           # app_auth + account_auth + refresh token (14 j)
│   ├── hb.rs             # sender 10s + watchdog 35s + reconnect backoff
│   ├── spot.rs           # SymbolsList → 28 symbolId → subscribe → ingest
│   ├── orders.rs         # LIMIT IOC séquentiel + clientMsgId correlation
│   ├── error.rs          # ProtoOAErrorCode mapping → actions typées
│   ├── metrics.rs        # HDR histogram latence + PnL compteur
│   └── main.rs           # modes : detect-only / demo / live
├── config/
│   ├── demo.toml         # host=demo.ctraderapi.com
│   └── live.toml         # host=live.ctraderapi.com
└── tests/
    └── integration.rs    # mock TCP server local
```

Figure 1 — Structure cible de la crate ctrader_client/

2.2 — Module conn.rs (connexion + codec)

Reprise quasi-intégrale du guide (chapitre 5) avec les deux corrections : (1) endpoints séparés démo/live via l'enum `Env`, (2) timeout de connexion explicite 5 s + TCP_NODELAY obligatoire. Le codec de tramage 4 octets Big-Endian + payload est inchangé (correct dans le guide).

RUST · SRC/CONN.RS — SQUELETTE

```rust
// src/conn.rs — squelette (raccourcis voir guide chap. 5)
use tokio_rustls::client::TlsStream;
use tokio::net::TcpStream;
pub enum Env { Demo, Live }
impl Env {
    pub fn host(&self) -> &'static str {
        match self {
            Env::Demo => "demo.ctraderapi.com",  // CORRIGÉ (1.1)
            Env::Live => "live.ctraderapi.com",   // CORRIGÉ (1.1)
        }
    }
}
pub const PORT: u16 = 5035;
pub const MAX_FRAME_SIZE: usize = 1_048_576;
pub struct Connection {
    inner: Arc<Mutex<TlsStream<TcpStream>>>,
}
impl Connection {
    pub async fn connect(env: Env) -> Result<Self> {
        let addr = format!("{}:{}", env.host(), PORT);
        let tcp = tokio::time::timeout(
            std::time::Duration::from_secs(5),
            TcpStream::connect(addr.as_str()),
        ).await??;
        tcp.set_nodelay(true)?;  // CRITIQUE pour latence
        // ... handshake TLS rustls (cf guide chap. 5)
        unimplemented!()
    }
    pub async fn send_msg(&self, msg: &ProtoMessage) -> Result<()> {
        // Frame : 4 octets BE (taille) + payload protobuf
        // ... cf guide chap. 5
        unimplemented!()
    }
    pub async fn recv_msg(&self) -> Result<ProtoMessage> {
        // Lecture 4 octets BE → N → N octets payload → décodage
        // ... cf guide chap. 5
        unimplemented!()
    }
}
```

2.3 — Module auth.rs (auth 2 niveaux + refresh)

Reprise intégrale du guide (chapitre 6). L'authentification en deux étapes (2100 application → 2102 account) est inchangée. Le refresh token via `ProtoOARefreshTokenReq` (2173) doit être déclenché automatiquement à l'expiration du token (signalé via `ProtoOAErrorCode::OA_AUTH_TOKEN_EXPIRED`, code 1), ou préventivement 1 heure avant l'expiration théorique (14 jours par défaut pour IC Markets).

2.4 — Module hb.rs (heartbeat + watchdog)

Reprise intégrale du guide (chapitre 7) : sender 10 s + watchdog 35 s + reconnect backoff exponentiel (1 s → 2 s → 4 s → max 60 s avec jitter). Le circuit breaker (5 reconnects en 1 min → pause 5 min) est inclus.

2.5 — Module spot.rs (28 paires FX)

Adaptation du guide (chapitre 8) pour **28 paires FX** au lieu de 3. Les `symbolId` sont dynamiques (récupérés via `ProtoOASymbolsListReq` au démarrage) car ils peuvent varier entre comptes et dans le temps. La structure `SpotManager` expose une `DashMap` indexée par symbolId, et un canal broadcast pour notifier l'engine.

RUST · SRC/SPOT.RS — 28 PAIRES

```rust
// src/spot.rs — adaptation 28 paires
pub const PAIRS_FX: &[&str] = &[
    "EURUSD", "USDJPY", "GBPUSD", "USDCHF", "USD CAD", "AUDUSD", "NZDUSD",
    "EURGBP", "EURJPY", "EURCHF", "EURCAD", "EURAUD", "EURNZD",
    "GBPJPY", "GBPCHF", "GBPCAD", "GBPAUD", "GBPNZD",
    "CHFJPY", "CADJPY", "CADCHF", "AUDJPY", "AUDCAD", "AUDCHF", "AUDNZD",
    "NZDJPY", "NZDCAD", "NZDCHF",
    // 28 paires — adapter selon IC Markets
];
pub async fn subscribe_all(
    conn: &Connection,
    ctid_account_id: i64,
    symbols: &[ProtoOASymbolByIdRes],
) -> Result<Vec<i64>> {
    let mut ids = Vec::with_capacity(PAIRS_FX.len());
    for name in PAIRS_FX {
        if let Some(sym) = symbols.iter().find(|s| s.symbol_name.as_deref() == Some(*name)) {
            ids.push(sym.symbol_id);
        } else {
            tracing::warn!(pair = %name, "symbole introuvable sur ce compte");
        }
    }
    // Subscribe par batches de 20 (limite côté serveur)
    for chunk in ids.chunks(20) {
        subscribe_spots(conn, ctid_account_id, chunk).await?;
    }
    Ok(ids)
}
```

2.6 — Module orders.rs (LIMIT IOC séquentiel)

Remplace le module du guide (chapitre 9) qui envoyait des ordres MARKET en parallèle. Ici : ordres `LIMIT IOC` séquentiels par jambe, avec un prix calculé à `touche ± 3·τ_slip` (où τ_slip est le slippage toléré, calibré sur backtest). L'ordre IOC annule automatiquement la part non remplie, ce qui permet à l'engine de compenser les jambes non exécutées via le mécanisme `exec_settle` (Convention A résiduel).

RUST · SRC/ORDERS.RS — LIMIT IOC

```rust
// src/orders.rs — LIMIT IOC séquentiel
use crate::proto::spotware::{
    ProtoOANewOrderReq, ProtoOAOrderType, ProtoOATradeSide,
    ProtoOATimeInForce, ProtoOAPayloadType,
};
pub const TAU_SLIP_PIPS: f64 = 0.5;  // calibré sur backtest
pub const LIMIT_OFFSET_PIPS: f64 = 3.0 * TAU_SLIP_PIPS;  // 1.5 pip
pub async fn send_limit_ioc(
    conn: &Connection,
    ctid_account_id: i64,
    symbol_id: i64,
    side: TradeSide,
    volume_lots: f64,
    decimals: u8,
    ref_price: f64,        // prix de référence (mid)
```

```
    client_msg_id: &str,
) -> Result<()> {
    // Prix LIMIT = touche ± 3·τ_slip
    let pip = 10f64.powi(-(decimals as i32 - 1));
    let limit_price = match side {
        TradeSide::BUY  => ref_price + LIMIT_OFFSET_PIPS * pip,
        TradeSide::SELL => ref_price - LIMIT_OFFSET_PIPS * pip,
    };
    let req = ProtoOANewOrderReq {
        ctid_trader_account_id: ctid_account_id,
        symbol_id,
        order_type: ProtoOAOrderType::LIMIT,
        trade_side: side,
        volume: lots_to_volume(volume_lots),  // CORRIGÉ (1.2) : 1 lot = 10_000_000
        limit_price: Some(limit_price),
        time_in_force: Some(ProtoOATimeInForce::IMMEDIATE_OR_CANCEL as i32),
        // Pas d'expirationTimestamp pour IOC (annulation immédiate)
        comment: Some(format!("ioc-{}", client_msg_id)),
        ..Default::default()
    };
    let msg = build_msg(
        ProtoOAPayloadType::PROTO_OA_NEW_ORDER_REQ as i32,
        &req, Some(client_msg_id));
    conn.send_msg(&msg).await
}
```

2.7 — Module metrics.rs (latence + PnL)

Instrumentation minimale mais exhaustive sur les dimensions critiques : (1) **latence** p50/p95/p99 via HDR Histogram pour les étapes tick-reçu → détection → envoi-ordre ; (2) **PnL** cumulé par session et par opportunité, exposé via un compteur atomique. Pas de Prometheus/Grafana dans cette première phase (ajoutable plus tard via `prometheus_exporter`).

RUST · SRC/METRICS.RS

```
// src/metrics.rs — latence + PnL
use hdrhist::Histogram;
use std::sync::Mutex;
use once_cell::sync::Lazy;
static LATENCY_TICK_TO_ORDER: Lazy<Mutex<Histogram<u64>>> = Lazy::new(|| {
    Mutex::new(Histogram::new_with_bounds(1, 10_000_000, 3).unwrap())
});
static PNL_CUMULATED: std::sync::atomic::AtomicI64 =
    std::sync::atomic::AtomicI64::new(0);  // en centimes d'EUR
pub fn record_tick_to_order_latency(start: std::time::Instant) {
    let elapsed_us = start.elapsed().as_micros() as u64;
    LATENCY_TICK_TO_ORDER.lock().unwrap().record(elapsed_us).ok();
}
pub fn add_pnl(pnl_eur: f64) {
    let cents = (pnl_eur * 100.0).round() as i64;
    PNL_CUMULATED.fetch_add(cents, std::sync::atomic::Ordering::Relaxed);
}
pub fn snapshot() -> MetricsSnapshot {
    let h = LATENCY_TICK_TO_ORDER.lock().unwrap();
    MetricsSnapshot {
        latency_p50_us: h.value_at_quantile(0.5),
        latency_p95_us: h.value_at_quantile(0.95),
        latency_p99_us: h.value_at_quantile(0.99),
        latency_max_us: h.max(),
        pnl_cumulated_eur: PNL_CUMULATED.load(
            std::sync::atomic::Ordering::Relaxed) as f64 / 100.0,
    }
}
```

2.8 — Module main.rs (3 modes)

Trois modes d'exécution strictement séparés, sélectionnés par flag CLI :

Mode	Description	Ordres ?	Volume
detect-only	Shadow : log uniquement les opportunités détectées	Aucun	N/A
demo	Ordres réels sur compte démo IC Markets	OUI (LIMIT IOC)	0.01 lot
live	Production sur compte réel	OUI (LIMIT IOC)	progressif 0.01 → 1.0

Tableau 4 — Modes d'exécution de ctrader_client/

Le passage detect-only → demo → live est **manuel et explicite** : il requiert un flag `--mode demo` (ou `--mode live`) sur la ligne de commande, plus une variable d'environnement `CTRADER_CONFIRM_LIVE=1` pour le mode live. Cette double confirmation évite tout démarrage accidentel en live.

SECTION 3 · MAPPING GUIDE → PROD

Ce qu'on garde, ce qu'on jette

Table de décision formelle entre les éléments du guide principal (42 pages) et le cahier des charges `ctrader_client/`. Cette table fait foi en cas de divergence entre les deux documents.

Élément du guide	Décision	Notes
Codec trame 4 octets BE + recv_msg/send_msg (chap. 5)	✅ Gardé tel quel	Conn.rs — vérifié, correct
Auth 2 étages 2100→2102 + clientMsgId + refresh 2173 (chap. 6)	✅ Gardé tel quel	auth.rs — pattern oneshot inchangé
Heartbeat sender 10s + watchdog 35s (chap. 7)	✅ Gardé tel quel	hb.rs — vérifié sur forum cTrader
Reconnect backoff + circuit breaker (chap. 13)	✅ Gardé tel quel	hb.rs — backoff 1s→60s + jitter
SymbolsList 2114 → subscribe spots 2127 (chap. 8)	✅ Adapté 28 paires	spot.rs — symbolId dynamiques, batches de 20
SpotManager DashMap + broadcast (chap. 8)	✅ Gardé tel quel	spot.rs — indexé par symbolId
OrderManager + ExecutionEvent 2126/2132/2142 (chap. 9)	✅ Gardé structure	orders.rs — mais ordres LIMIT IOC, pas MARKET
Reconcile 2124 (chap. 11)	✅ Gardé tel quel	orders.rs — cohérence état mémoire/serveur
Slippage guard 2 pips (chap. 13)	✅ Gardé tel quel	error.rs — cooldown 60s si dépassé
Exemple arbitrage triangulaire + tokio::join! 3 ordres (chap. 11)	❌ Ignoré	Pas notre sémantique — chemin ouvert E1-E9 séquentiel
Ordres MARKET parallèles (chap. 9, 11)	❌ Remplacé	LIMIT IOC séquentiel par jambe (parité backtest)
Cycle EUR→USD→CHF→EUR 3 paires (chap. 10)	❌ Hors scope	Engine principal gère les cycles, pas ctrader_client/
VPS LD5 Equinix (chap. 12)	⏸ Ignoré	Déploiement sur serveur actuel pour la démo
CPU pinning + lock-free crossbeam (chap. 12)	⏸ Plus tard	Optimisations HFT reportées à la phase live
Volume « 100 000 = 1 lot » (chap. 9 tableau 5)	❌ CORRIGÉ ×100	1 lot = 10 000 000 — voir section 1.2
Endpoint ctraderapi.com:5035 (chap. 2)	❌ CORRIGÉ	demo/live.ctraderapi.com:5035 — voir section 1.1

Élément du guide	Décision	Notes
Heartbeat ID 51 (chap. 2, 7)	✓ Correct	Aucune correction nécessaire

Tableau 5 — Mapping formel guide → ctrader_client/

SECTION 4 · PLAN DE DÉMARRAGE

Plan de démarrage en 3 phases

Le démarrage respecte une séquence stricte à trois phases, avec des critères de passage (*gate criteria*) explicites entre chaque phase. Aucune phase ne peut être franchie sans validation manuelle des critères.

Phase	Durée	Mode	Volume	Gate pour passer à la suite
1 — detect-only	48-72 h	detect-only	N/A (zéro ordre)	≥ 5 opportunités/jour détectées, latence p99 < 5 ms, zéro crash
2 — démo ordres réels	5 jours ouvrés	demo	0.01 lot (1k unités)	Win rate > 50%, profit factor > 1.2, zéro erreur critique
3 — live progressif	Indéfini	live	0.01 → 0.1 → 1.0	PnL net positif sur 7 jours consécutifs avant montée en volume

Tableau 6 — Plan de démarrage à 3 phases

4.1 — Phase 1 : detect-only (48-72 h)

Le moteur tourne en mode **shadow** : il écoute les ticks, calcule les opportunités d'arbitrage, mais n'envoie **aucun ordre**. Le but est de calibrer `net_min` (seuil de rentabilité net) et de mesurer le taux réel d'opportunités par jour en condition de marché live. Sans cette phase, on risque de coder un seuil trop optimiste qui ne tient pas face aux spreads réels d'IC Markets.

- Critère 1 : au moins 5 opportunités/jour détectées avec un PnL théorique > 2,5 USD par lot.
- Critère 2 : latence p99 (tick → détection) < 5 ms, sinon optimiser le SpotManager ou le dispatch loop.
- Critère 3 : zéro crash, zéro déconnexion non-récupérée sur 24 h.

4.2 — Phase 2 : démo ordres réels (5 jours ouvrés)

Activation des ordres LIMIT IOC sur compte démo IC Markets. Volume **0.01 lot** (1 000 unités) par jambe — suffisant pour valider le fill sans risquer de capital significatif. La session de trading est limitée à **07:00-21:00 heure de Zurich** (couverture Londres + New York) selon le cahier des charges initial.

- Critère 1 : **win rate > 50%** sur les opportunités exécutées.
- Critère 2 : **profit factor > 1.2** (somme gains / somme pertes).
- Critère 3 : zéro erreur critique (NOT_ENOUGH_MONEY, TRADING_DISABLED, circuit breaker déclenché plus de 2 fois).
- Critère 4 : slippage moyen < 0.5 pip par jambe (sinon recalibrer `TAU_SLIP_PIPS`).

4.3 — Phase 3 : live progressif

Passage en live avec montée en volume graduelle : 0.01 lot → 0.1 lot → 1.0 lot. Chaque palier est maintenu minimum 7 jours avant la montée suivante, et requiert un PnL net positif sur cette période.

4.4 — Prérequis obligatoires avant la phase 1

> **ACTION REQUISE · CRÉATION DE L'APP CTRADER**
> Avant tout test, créer l'application sur **openapi.ctrader.com** et récupérer : (1) **clientId**, (2) **clientSecret**, (3) **accessToken** lié au compte démo IC Markets, (4) **ctidTraderAccountId** (visible dans cTrader > Account > Details). Sans ces 4 identifiants, aucune connexion n'est possible.

SECTION 5 · QUESTIONS À CONFIRMER

Décisions en attente de validation

Trois questions techniques restent à confirmer avant le démarrage de la phase 1. Toutes les autres sont tranchées par le présent addendum.

Q1 — LIMIT IOC confirmé ?

Le choix **LIMIT IOC** seul (sans variante MARKET_RANGE) est acté par défaut pour préserver la parité avec le backtest validé. **Confirmé ?** Si oui, le module `orders.rs` est figé tel que présenté en section 2.6.

Q2 — Démarrage en detect-only ?

Le démarrage en mode detect-only pendant 48-72 h avant tout ordre réel est la recommendation par défaut. **OK pour démarrer ainsi ?** Si oui, le mode sera codé en priorité dans `main.rs` avec un logger dédié qui enregistre chaque opportunité détectée dans `/var/log/utilustrader/detect.log` au format JSON structuré.

Q3 — Volume démo 0.01 lot ?

Volume démo de 0.01 lot (1 000 unités) par jambe. C'est le minimum pratique sur IC Markets (le broker accepte 0.001 mais le spread devient proportionnellement trop coûteux). **Confirmez-vous 0.01 lot ?**

---

*Addendum au guide **ctrader-rust-arbitrage-guide.pdf** — corrige les sections 1.1 (endpoints), 1.2 (volume ×100), 1.3 (LIMIT IOC) et formalise le cahier des charges de la crate ctrader_client/. En cas de divergence entre ce document et le guide principal, l'addendum fait foi. · Version 1.0 · 2026-11.*